

Contents

Framework-Walkthrough (Tag 1, 09:15–10:45)	1
0. Vorab: die drei Konzepte, die im Framework wichtig sind	1
1. Live-Demo zuerst (5 Min)	3
2. Das Domain-Modell — <code>framework/arena/Models.kt</code> (56 Zeilen)	3
3. Wie wird <code>decide()</code> aufgerufen? — <code>GameEngine.step()</code>	7
4. Absicherung gegen kaputten Bot-Code — <code>BotExecutor</code> (2 Min, nur konzeptuell)	7
5. Ein fertiger Beispiel-Bot — <code>bots/examples/RandomBot.kt</code> (23 Zeilen)	8
6. Wo trage ich meinen Code ein? — <code>bots/teamA/TeamABots.kt</code>	8
Vorschlag Zeitbudget (90 Min gesamt, 09:15–10:45)	9
Was bewusst NICHT gezeigt wird (und warum)	10

Framework-Walkthrough (Tag 1, 09:15–10:45)

Begleitmaterial für den dozentengeführten Block aus [tag-1.md](#). Diese Datei erklärt nicht nur *was* gezeigt wird, sondern auch *wie man es erklärt* — inklusive der Grundkonzepte (Interface, Enum, ...), falls die im Vorlauf (Tag 1/2 Kotlin-Grundlagen) nur gestreift wurden.

Leitsatz für den Block: Schüler schreiben ausschließlich den Body von `RobotBrain.decide()`. Alles hier Gezeigte ist fertig und wird nicht verändert — Ziel ist Verständnis, nicht Bearbeitung.

0. Vorab: die drei Konzepte, die im Framework wichtig sind

Bevor der eigentliche Code gezeigt wird, hier die drei Kotlin-Konzepte, die im Framework ständig vorkommen — mit einfachen Erklärungen, die man 1:1 an die Schüler weitergeben kann. Wer die Konzepte schon sicher beherrscht, kann diesen Abschnitt überspringen und direkt zu Abschnitt 1 gehen.

Was ist ein Interface?

Ein Interface ist ein **Vertrag**: es legt fest, *welche* Funktionen und Eigenschaften eine Klasse haben muss, aber nicht *wie* sie das macht.

Analogie: Ein Interface ist wie eine Stellenausschreibung. “Gesucht: jemand, der kochen kann.” Die Ausschreibung sagt nicht, *wie* gekocht wird — nur, dass die Person die Fähigkeit “kochen” mitbringen muss. Jede Person, die sich bewirbt (= jede Klasse, die das Interface implementiert), kocht vielleicht ganz anders, aber alle erfüllen den Vertrag “kann kochen”.

```
interface RobotBrain {  
    val name: String  
    fun decide(sensors: Sensors): Action  
}
```

Das bedeutet: “Jede Klasse, die `RobotBrain` sein will, MUSS einen `name` und eine Funktion `decide(sensors)` haben, die eine `Action` zurückgibt.” *Wie* die Funktion entscheidet, ist komplett offen — das ist genau der Teil, den die Schüler selbst schreiben.

Warum ist das nützlich? Die Engine (der Rest des Frameworks) muss nicht wissen, ob sie gerade mit `RandomBot`, `ChaserBot` oder dem selbstgeschriebenen Team-Bot spricht. Sie weiß nur: “Das ist ein `RobotBrain`, also hat es `decide()`, also kann ich es aufrufen.” Das ist der ganze Trick, mit dem drei komplett unterschiedliche Team-Bots im selben Spiel gegeneinander antreten können, ohne dass die Engine für jeden Bot extra angepasst werden muss.

Was ist ein Enum (enum class)?

Ein Enum ist eine **feste, abzählbare Liste von Werten**, die es geben darf — nichts anderes ist erlaubt.

Analogie: Eine Ampel kann nur Rot, Gelb oder Grün sein. Es gibt kein “ein bisschen Gelb” oder “Blau”. Ein Enum in Kotlin bildet genau so eine feste Auswahl ab.

```
enum class Direction(val dx: Int, val dy: Int) {  
    NORTH(0, -1), EAST(1, 0), SOUTH(0, 1), WEST(-1, 0)  
}
```

Hier gibt es genau vier mögliche Richtungen — `Direction.NORTH`, `.EAST`, `.SOUTH`, `.WEST`. Man kann nicht aus Versehen `Direction.NORTHWEST` schreiben, das würde gar nicht kompilieren. Das schützt vor Tippfehlern und ungültigen Werten.

Das `(val dx: Int, val dy: Int)` dahinter ist etwas Zusätzliches: **jeder** Wert des Enums trägt selbst noch zwei Zahlen mit sich herum. `dx/dy` beschreiben, wie sich die x/y-Koordinate verändert, wenn man einen Schritt in diese Richtung geht. Also: `Direction.NORTH.dx` ist `0`, `Direction.NORTH.dy` ist `-1`. Man kann sich das wie ein Enum mit eingebauten “Zusatzinfos” pro Wert vorstellen.

Stolperfalle, die man den Schülern vorab sagen sollte: In diesem Framework wächst die y-Koordinate nach unten (wie bei Bildschirm-Koordinaten, nicht wie im Matheunterricht). `NORTH` (nach oben auf dem Bildschirm) bedeutet deshalb `y - 1`, nicht `y + 1`. Das verwirrt fast immer beim ersten Mal.

Was ist eine data class?

Eine `data class` ist eine Klasse, die nur Daten zusammenhält (kein eigenes Verhalten), und für die Kotlin automatisch nützliche Funktionen mitliefert: Vergleichen zweier Objekte (`==`), eine lesbare Text-Darstellung, und eine Kopier-Funktion.

```
data class Position(val x: Int, val y: Int)
```

Das ist im Grunde nur “eine Position hat ein x und ein y”. Ohne `data` müsste man selbst Code schreiben, damit zwei `Position`-Objekte mit gleichen Werten auch als “gleich” erkannt werden. Mit `data` passiert das automatisch.

Was ist ein sealed interface?

Das ist die Steigerung von Interface + Enum: ein `sealed interface` legt fest, **welche festen, unterschiedlich aufgebauten Möglichkeiten** es für etwas gibt — anders als ein Enum, wo jeder Wert gleich aussieht (nur Name + evtl. Zusatzwerte), können bei einem `sealed interface` die einzelnen Möglichkeiten ganz unterschiedliche Daten enthalten.

```
sealed interface Action {  
    data class Move(val direction: Direction) : Action  
    data class Shoot(val direction: Direction) : Action  
    data object Wait : Action  
}
```

Eine `Action` ist **immer genau eine** von drei Dingen: - `Action.Move(direction)` — enthält eine Richtung, in die man sich bewegen will - `Action.Shoot(direction)` — enthält eine Richtung, in die man schießen will - `Action.Wait` — enthält gar nichts, einfach “nichts tun”

Der Unterschied zum Enum: `Move` und `Shoot` tragen zusätzlich eine `Direction` mit sich, `Wait` trägt gar nichts. Das wäre mit einem einfachen Enum nicht abbildbar.

Für die Schüler reicht: *“Ihr müsst aus eurer decide()-Funktion immer genau eines von diesen drei Dingen zurückgeben: Action.Move(Direction.NORTH), Action.Shoot(Direction.EAST), oder Action.Wait.”* Wie man selbst einen sealed interface definiert, ist **nicht** Lernziel — nur die Benutzung.

1. Live-Demo zuerst (5 Min)

./gradlew run starten, RandomBot vs. ChaserBot laufen lassen. Schüler sehen das Endergebnis, bevor auch nur eine Zeile Code gezeigt wird — Motivation vor Theorie.

Sprechzeile: *“Das hier ist das Endprodukt von heute. Zwei fertige Bots kämpfen gegeneinander. Am Ende des Tages soll euer eigener Bot genauso hier mitspielen.”*

Kurz auf UI zeigen, ohne Details: Arena links (10×10-Raster), Scoreboard und Log rechts. Nicht erklären, wie das gebaut ist — nur, dass es existiert und fertig ist.

2. Das Domain-Modell — framework/arena/Models.kt (56 Zeilen)

Datei komplett öffnen, im Editor von oben nach unten scrollen. Das ist die zentrale Datei — hier stehen alle “Bausteine”, mit denen ein Bot arbeitet.

Direction

```
enum class Direction(val dx: Int, val dy: Int) {  
    NORTH(0, -1), EAST(1, 0), SOUTH(0, 1), WEST(-1, 0)  
}
```

Siehe Erklärung “Was ist ein Enum” oben. Konkret zeigen: Direction.NORTH.dx ist 0, Direction.NORTH.dy ist -1 — Richtung nach oben auf dem Bildschirm.

Position

```
data class Position(val x: Int, val y: Int) {  
    fun moved(direction: Direction) = Position(x + direction.dx, y +  
        ↪ direction.dy)  
}
```

Konzept dahinter: Eine Position ist einfach ein Koordinatenpaar auf dem 10×10-Raster. (0, 0) ist oben links (typisch für Computergrafik, anders als im Matheunterricht üblich, wo (0,0) oft in der Mitte oder unten links liegt).

Warum ist das eine eigene Klasse und nicht einfach zwei lose Zahlen x und y? Weil man dann nie zwei zusammengehörige Werte getrennt herumreichen muss (“wo ist eigentlich das y zu diesem x?”) und weil Kotlin für data class automatisch Vergleiche mitliefert: zwei Position-Objekte mit gleichem x und y gelten mit == automatisch als gleich. Das wird später wichtig, z.B. um zu prüfen “steht der Gegner auf derselben Position wie ich?”

Die Funktion moved(direction) ist die praktische Anwendung des Enums von eben: “Gib mir die Position, die einen Schritt in diese Richtung liegt.” Beispiel an der Tafel: Roboter steht auf (3, 3), bewegt sich NORTH -> neue Position ist (3 + 0, 3 + (-1)) = (3, 2). Bewusst durchrechnen, das festigt “y wird kleiner bei NORTH” nochmal visuell.

Kurzer Hinweis zur Kurzschreibweise `fun moved(...) = Position(...)`: das ist nur eine kürzere Schreibweise für eine Funktion, die nur eine einzige Berechnung zurückgibt — entspricht `fun moved(...): Position { return Position(...) }`.

Konkretes Nutzungsbeispiel aus `decide()`: Angenommen, ein Bot will wissen, ob er sich gerade am linken Rand befindet:

```
val amLinkenRand = sensors.self.position.x == 0
```

Oder: die Distanz zu einem Gegner grob abschätzen (nur x-Differenz, ohne Wurzelziehen — für die Bot-Logik reicht das):

```
val gegner = sensors.others.first()
val distanzX = sensors.self.position.x - gegner.position.x
```

`sensors.self.position` ist der Einstiegspunkt für praktisch jede räumliche Entscheidung, die ein Bot trifft — Distanz zu Gegnern, Nähe zum Rand, Richtung zum Zentrum, etc. Diese Zeile lohnt sich, laut mitzusprechen: *“Jede Positions-Frage, die euer Bot beantworten will, fängt mit `sensors.self.position` an.”*

RobotState

```
data class RobotState(
    val id: String,
    val teamName: String,
    val position: Position,
    val health: Int
) {
    val alive: Boolean get() = health > 0
}
```

Konzept dahinter: Das ist der komplette “Steckbrief” eines Roboters zu einem bestimmten Zeitpunkt: eine ID (z.B. “bot-0”, wird von der Engine automatisch vergeben), ein Name, eine Position, und die aktuelle Gesundheit (Health Points, HP).

Warum ein `RobotState` und nicht einfach vier einzelne Werte? Weil man dann alles, was zu einem Roboter gehört, mit einer Variable herumreichen kann. `Sensors` enthält z.B. eine ganze `List<RobotState>` — ohne dieses Bündel müsste man vier separate Listen (eine für IDs, eine für Positionen, eine für Health, ...) synchron halten, was extrem fehleranfällig wäre.

`alive` genauer erklärt: Es steht **kein fester Wert** dahinter, sondern eine kleine Berechnung (`get()` = ...). Jedes Mal, wenn man `robotState.alive` abfragt, wird live geprüft: “ist `health` größer als 0?” Der Vorteil: man muss sich nie sorgen, ob `alive` “vergessen” wurde zu aktualisieren, wenn `health` sich ändert — es kann gar nicht aus dem Takt geraten, weil es nicht separat gespeichert wird, sondern jedes Mal frisch berechnet wird.

Konkretes Nutzungsbeispiel aus `decide()`: Die eigene Gesundheit abfragen, um zwischen Angriff und Flucht zu entscheiden (das ist praktisch Story 1.3 aus dem Backlog):

```
if (sensors.self.health < 20) {
    // fliehen
} else {
    // normal weiterkämpfen
}
```

Oder: nur noch lebende Gegner betrachten, die eine bestimmte Bedingung erfüllen, z.B. den mit der niedrigsten Gesundheit als Ziel wählen (Story 3.2):

```
val schwachsterGegner = sensors.others.minByOrNull { it.health }
```

`sensors.self` ist immer ein `RobotState` — der eigene. Jeder Eintrag in `sensors.others` ist ebenfalls ein `RobotState` — aber von einem Gegner. Das ist derselbe Bauplan, nur für unterschiedliche Roboter befüllt.

Sensors

```
data class Sensors(  
    val self: RobotState,  
    val others: List<RobotState>,  
    val arenaWidth: Int,  
    val arenaHeight: Int,  
    val tick: Int  
)
```

Konzept dahinter: `Sensors` ist **das einzige Fenster zur Welt**, das ein Bot hat. Es wird von der Engine **jeden Tick neu gebaut** und an `decide()` übergeben — der Bot bekommt also bei jedem Aufruf einen druckfrischen Schnappschuss des aktuellen Spielstands, nie einen veralteten.

Analogie: Stellt euch vor, ihr spielt Schach, aber mit verbundenen Augen — jemand sagt euch einmal pro Zug genau, wo alle Figuren gerade stehen, und dann müsst ihr sofort entscheiden. Ihr könnt euch nicht “vorher schon gemerkt haben, wo der Gegner letzten Zug war” — es zählt nur, was gerade in `Sensors` drinsteht.

Enthält genau fünf Dinge — wichtig, das explizit durchzugehen, das ist buchstäblich **alles**, was ein Bot wissen darf:

- `self` — der eigene `RobotState` (eigene Position, eigene Health, eigene ID)
- `others` — eine `List<RobotState>` aller **noch lebenden** anderen Roboter. Tote Roboter tauchen hier nicht mehr auf — ein Bot muss also nie selbst prüfen, ob ein Gegner schon tot ist, das hat die Engine schon für ihn erledigt.
- `arenaWidth/arenaHeight` — Größe des Spielfelds (beides 10, aber bewusst nicht fest im Code der Bots verdrahtet, sondern übergeben — falls sich die Arena-Größe mal ändert, muss kein Bot-Code angepasst werden)
- `tick` — die aktuelle Runden-Nummer, falls ein Bot zeitabhängiges Verhalten will (z.B. “in den ersten 10 Ticks patrouillieren, danach angreifen”)

Warum so eingeschränkt — kein Zugriff auf die Engine selbst, keine Erinnerung an vorherige Ticks? Zwei Gründe, die man den Schülern nennen kann: 1. **Fairness:** Kein Bot kann “schummeln”, indem er auf interne Engine-Daten zugreift oder Dinge sieht, die er im echten Spiel nicht sehen dürfte. 2. **Einfachheit:** `decide()` muss nur eine Frage beantworten — “was tue ich JETZT, basierend auf dem, was ich JETZT sehe?” Kein Bot muss sich selbst Zustand über mehrere Ticks hinweg merken, es sei denn er will es (das ist optional möglich über eine eigene `var` in der Bot-Klasse, siehe Story 3.1 im Backlog).

Konkretes Nutzungsbeispiel — ein kompletter Mini-Bot, der alle vier Sensor-Felder mindestens einmal benutzt:

```
override fun decide(sensors: Sensors): Action {  
    // Gibt es überhaupt noch einen Gegner?  
    val gegner = sensors.others.firstOrNull() ?: return Action.Wait  
  
    // Stehe ich mit dem Gegner in derselben Zeile (gleiches y)?  
    return if (sensors.self.position.y == gegner.position.y) {  
        Action.Shoot(Direction.EAST)  
    } else {  
        Action.Move(Direction.SOUTH)  
    }  
}
```

```
}
```

Diese vier Zeilen sind fast eine Vorschau auf Story 2.2 aus dem Backlog (Zielen auf Gegner in Sichtlinie) — gut geeignet, um zu zeigen, dass die Storys später genau auf diesen Bausteinen aufbauen.

Action

```
sealed interface Action {  
    data class Move(val direction: Direction) : Action  
    data class Shoot(val direction: Direction) : Action  
    data object Wait : Action  
}
```

Konzept dahinter: Siehe ausführliche Erklärung “Was ist ein sealed interface” oben — hier nochmal konkret auf diesen Anwendungsfall bezogen. Action ist die **einzige Art, wie ein Bot mit der Spielwelt kommunizieren kann**. Ein Bot kann nichts direkt verändern (nicht selbst seine position setzen, nicht selbst health abziehen) — er kann nur eine Action *vorschlagen*, und die Engine setzt sie dann nach ihren eigenen Regeln um (inklusive der Konfliktregeln aus Abschnitt 3, z.B. wenn zwei Bots ins selbe Feld wollen).

Das ist ein wichtiges Prinzip, das sich lohnt laut auszusprechen: *“Euer Bot schlägt vor, die Engine entscheidet.”* Ein Bot kann z.B. Action.Move(Direction.NORTH) zurückgeben, obwohl direkt nördlich eine Wand oder ein anderer Roboter ist — die Engine prüft das und lässt die Bewegung im Zweifel einfach ins Leere laufen. Der Bot bekommt dafür keine Fehlermeldung, es passiert einfach nichts.

Warum drei Möglichkeiten und nicht z.B. drei einzelne Funktionen move(), shoot(), wait()? Weil decide() **genau eine** Aktion pro Tick zurückgeben muss — nie zwei gleichzeitig (nicht “bewege dich UND schieße”). Der sealed interface-Typ erzwingt das automatisch: die Funktion hat den Rückgabebetyp Action, und Action kann eben immer nur eines der drei Dinge gleichzeitig sein.

Konkrete Nutzungsbeispiele, alle drei Fälle:

```
Action.Move(Direction.NORTH)    // einen Schritt nach oben gehen  
Action.Shoot(Direction.WEST)    // nach links schießen  
Action.Wait                     // nichts tun (kein Klammern, da kein Parameter)
```

Kleiner, aber wichtiger Stolperstein: Action.Wait wird **ohne** Klammern geschrieben (Action.Wait, nicht Action.Wait()), weil es ein data object ist — ein Singleton, kein Konstruktor-Aufruf. Move und Shoot dagegen brauchen Klammern mit einer Direction drin, weil es data classes sind, die einen Wert mitbekommen. Dieser Unterschied verwirrt fast immer beim ersten Schreiben — kurz explizit erwähnen.

RobotBrain

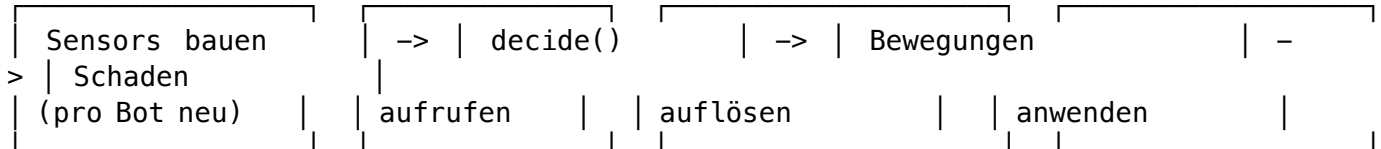
```
interface RobotBrain {  
    val name: String  
    fun decide(sensors: Sensors): Action  
}
```

Siehe ausführliche Erklärung “Was ist ein Interface” oben. Das ist der Vertrag, den jeder Bot erfüllen muss. **Das ist die einzige Sache, die Schüler wirklich selbst bauen: eine Klasse, die diesen Vertrag erfüllt.**

Zusammenfassender Satz für die Tafel: *“Ein Bot ist nichts anderes als ein Name plus eine Funktion, die aus Sensors eine Action macht.”*

3. Wie wird `decide()` aufgerufen? — `GameEngine.step()`

Jetzt bewusst **weg vom Code, hin zur Tafel/Whiteboard** — als Kontrastmoment. Die Datei `GameEngine.kt` selbst ist mit 285 Zeilen zu dicht und nutzt fortgeschrittene Kotlin-Konzepte (Higher-Order-Functions, `groupBy`, Destructuring), die für Kotlin-Anfänger noch zu viel wären. Stattdessen den Ablauf **als Diagramm** an die Tafel zeichnen:



Erklärung Schritt für Schritt (als Sprechtext, nicht an die Tafel schreiben):

1. **Für jeden lebenden Bot** wird ein frisches `Sensors`-Objekt gebaut — mit dem aktuellen Stand von allem.
2. `decide()` wird **einmal pro Bot pro Tick** aufgerufen. Nie öfter, nie für zwei Bots gleichzeitig vermischt.
3. **Alle** Antworten (Move/Shoot/Wait) von **allen** Bots werden erst gesammelt, bevor überhaupt etwas passiert.
4. Erst danach werden alle Bewegungen gleichzeitig aufgelöst, dann alle Schüsse gleichzeitig ausgewertet.

Wichtig zu betonen: Ein “Tick” ist eine Zeiteinheit im Spiel, in der *alle* Bots gleichzeitig einmal entscheiden dürfen — wie eine Runde bei einem Brettspiel, nur dass alle Spieler gleichzeitig ziehen statt nacheinander.

Zwei Aha-Momente, die man als Geschichte erzählen sollte (nicht als Regel)

Diese zwei Dinge erklären scheinbar “komische” Spielsituationen, die Schüler später beim Testen beobachten werden. Am besten als kleine Geschichte erzählen, nicht als trockene Regel:

Geschichte 1 — der Bewegungs-Konflikt: *“Stellt euch vor, zwei Bots stehen nebeneinander und wollen beide ins selbe freie Feld ziehen. Was passiert? Antwort: keiner von beiden bewegt sich in diesem Tick. Das Spiel entscheidet nicht per Zufall, wer zuerst darf — es lässt einfach beide stehen. Wenn ihr also später seht, dass euer Bot plötzlich ‘hängen bleibt’, obwohl das Feld vor ihm frei aussah: vielleicht wollte ein Gegner im selben Moment auf dasselbe Feld.”*

Geschichte 2 — der gesammelte Schaden: *“Alle Schüsse in einem Tick werden gesammelt und dann gemeinsam ausgewertet — nicht einer nach dem anderen. Das heißt: Wenn zwei Bots im selben Tick auf denselben dritten Bot schießen, und der dadurch stirbt, dann zählen trotzdem **beide** Schüsse noch, egal wer ‘zuerst’ geschossen hätte. Es gibt kein ‘zu spät, der ist schon tot’ innerhalb desselben Ticks.”*

Warum das so gebaut ist (nur bei Nachfrage erklären, nicht von selbst vertiefen): Sonst würde das Ergebnis eines Spiels davon abhängen, in welcher Reihenfolge die Engine zufällig die Bots intern abarbeitet — das wäre unfair und nicht reproduzierbar.

4. Absicherung gegen kaputten Bot-Code — `BotExecutor` (2 Min, nur konzeptuell)

Kein Code zeigen, nur die Kernaussage — das nimmt den Schülern die Angst, beim Ausprobieren “das ganze Spiel kaputt zu machen”:

“Wenn euer Bot-Code einen Fehler hat — eine Endlosschleife, eine Exception, egal was — dann crasht nicht das ganze Programm. Euer Bot macht in diesem Fall einfach gar nichts (Wait), bis ihr

den Fehler behoben habt. Ihr könnt also mutig ausprobieren, ohne dass ihr den anderen Teams das Spiel kaputt macht.”

Falls jemand nachfragt, wie das technisch geht: “Jeder Bot-Aufruf läuft in einem eigenen kurzen Zeitfenster. Wenn er dieses Fenster verpasst oder abstürzt, wird das abgefangen.” Mehr Tiefe ist für den Rahmen nicht nötig.

5. Ein fertiger Beispiel-Bot — bots/examples/RandomBot.kt (23 Zeilen)

Das ist die zentrale, aktive Übung des Blocks. Gemeinsam Zeile für Zeile am Beamer, Schüler sollen raten, was die nächste Zeile wohl tut, **bevor** sie erklärt wird:

```
class RandomBot(override val name: String = "RandomBot") : RobotBrain {
    override fun decide(sensors: Sensors): Action {
        val direction = Direction.entries.random()
        return if (Random.nextBoolean()) Action.Move(direction) else
            ↪ Action.Shoot(direction)
    }
}
```

Zeile für Zeile:

- `class RandomBot(...) : RobotBrain` — das `:` `RobotBrain` heißt “diese Klasse erfüllt den `RobotBrain`-Vertrag von eben”. Wenn hier `name` oder `decide()` fehlen würden, würde das Programm gar nicht kompilieren — Kotlin zwingt einen, den Vertrag vollständig zu erfüllen.
- `override val name: String = "RandomBot"` — `override` heißt “ich erfülle hiermit das, was das Interface von mir verlangt hat”. Der Bot heißt standardmäßig “RandomBot”.
- `Direction.entries` — das ist die Liste aller vier Enum-Werte (`[NORTH, EAST, SOUTH, WEST]`). `.random()` pickt zufällig einen davon aus.
- `if (Random.nextBoolean()) Action.Move(direction) else Action.Shoot(direction)` — eine Münzwurf-Entscheidung: 50% Bewegen, 50% Schießen, jeweils in die zufällig gewählte Richtung.

Wichtiger Kotlin-Hinweis: `if/else` wird hier nicht als reine Verzweigung benutzt (wie “wenn X, mach A, sonst mach B”), sondern gibt selbst einen **Wert** zurück, der direkt hinter `return` steht. Das ist in Kotlin normal, aber für Einsteiger aus anderen Sprachen oft neu — kurz erwähnen.

Optional, nur wenn Zeit bleibt: `ChaserBot.kt` zeigen (sucht nächstgelegenen Gegner über Distanzberechnung). Das ist aber schon Story-2.3-Niveau — nicht erzwingen, wenn die Zeit knapp wird.

6. Wo trage ich meinen Code ein? — bots/teama/TeamABots.kt

Das eigentliche Ziel des ganzen Blocks: jeder weiß danach genau, wo er/sie schreibt.

```
package bots.teama
```

```
import framework.arena.Action
import framework.arena.Direction
import framework.arena.RobotBrain
import framework.arena.Sensors
```

```
class MeinBot(override val name: String = "Team A – MeinBot") : RobotBrain {
    override fun decide(sensors: Sensors): Action {
```



```

// TODO: Bewege dich stattdessen in Richtung des nächsten Gegners
// TODO: Schieße wenn ein Gegner in Sichtlinie ist
// TODO: Reagiere auf niedrige eigene HP (sensors.self.health), z.B.
    ↪ mit Flucht
return Action.Move(Direction.SOUTH)
}
}

```

```
val teamABots: List<RobotBrain> = listOf(MeinBot())
```

Live demonstrieren: `Action.Move(Direction.SOUTH)` zu `Action.Move(Direction.entries.random())` ändern, `./gradlew run` neu starten, zeigen dass der Bot jetzt zufällig läuft. Das ist der Moment, an dem jedes Team merkt: *“So einfach ist der erste Schritt.”*

Der mit Abstand häufigste Compile-Fehler, den man vorab ankündigen sollte: eine neue Bot-Klasse wird angelegt, aber vergessen, sie unten in die `teamXBots`-Liste einzutragen (`listOf(MeinBot())`) — der Bot kompiliert dann zwar, taucht aber nicht in der App-Auswahl auf. Das explizit vorab sagen, damit Schüler das beim ersten Mal selbst erkennen statt zu verzweifeln.

Vorschlag Zeitbudget (90 Min gesamt, 09:15–10:45)

Zeit	Dauer	Inhalt
09:15–09:20	5 Min	Live-Demo (RandomBot vs. ChaserBot)
09:20–09:30	10 Min	Grundkonzepte: Interface, Enum, data class, sealed interface (Abschnitt 0)
09:30–09:55	25 Min	<code>Models.kt</code> komplett durchgehen, inkl. Beispiele je Typ (Abschnitt 2)
09:55–10:05	10 Min	<code>GameEngine.step()</code> als Tafel-Diagramm (Abschnitt 3)
10:05–10:08	3 Min	BotExecutor-Sicherheitsnetz, nur Kernaussage (Abschnitt 4)
10:08–10:25	17 Min	<code>RandomBot.kt</code> gemeinsam Zeile für Zeile (Abschnitt 5)
10:25–10:40	15 Min	<code>TeamXBots.kt</code> — wo trage ich ein, Live-Beispiel bauen und laufen lassen (Abschnitt 6)
10:40–10:45	5 Min	Fragen, Übergang zu Arbeitsblock 2 (Story 1.1)

Wenn die Grundkonzepte (Abschnitt 0) aus dem Kotlin-Vorlauf schon sicher sitzen, kann dieser Programmpunkt entfallen — die gewonnenen 10 Minuten dann in Abschnitt 2 (mehr Zeit für die Beispiele je Modell-Typ) oder Abschnitt 5 investieren. Die Nutzungsbeispiele in Abschnitt 2 (bei `Position`, `RobotState`, `Sensors`, `Action`) sind bewusst so gewählt, dass sie schon kleine Vorschauen auf spätere Backlog-Stories sind (z.B. Flucht bei niedriger Gesundheit, Zielen auf Gegner) — bei Zeitdruck lieber diese Beispiele kürzen als ganz weglassen, sie sind der Teil, der Schülern später beim eigenständigen Programmieren am meisten hilft.

Was bewusst NICHT gezeigt wird (und warum)

- **BotExecutor, resolveMoves()/resolveShots(), App.kt im Detail** — diese Dateien nutzen fortgeschrittene Kotlin-Features, die für einen 90-Minuten-Einstieg zu viel wären: Java-Interop mit Threads (BotExecutor), Higher-Order-Functions wie groupBy/mapNotNull (GameEngine), und Compose-spezifische Konzepte wie remember/LaunchedEffect (App.kt). Bei Nachfragen reicht: *“Das ist fertiges Framework, das müsst ihr nie anfassen — schaut’s euch gerne selbst an, wenn ihr neugierig seid, aber es ist nicht Teil der Aufgabe.”*
- **Wie man selbst ein Interface, einen Enum oder einen sealed interface definiert** — Lernziel ist nur, diese Konzepte zu *erkennen* und zu *benutzen* (z.B. `Action.Move(Direction.NORTH)` schreiben), nicht selbst neue davon zu bauen.